

Meeting 16

I think i missed the boundary condition on the boundary of the obstacle and this has to be fixed before fixing the problem.

```
%-----Domain Parameter-----%
% Grid Points
global Nx; Nx = 60 + 6;
global Ny; Ny = 200 + 6;

%--Important Parameter
INFINITE = 10^12;
eta = 10^-6;
error_eta = 10^-9;
CFL = 0.5;

% To track the error for each iterations
iteration_error = [];

% Dimension of Domain
Lx = 6;
Ly = 20;

% Domain
x = linspace(0,Lx,Nx);
y = linspace(0,Ly,Ny);
[X,Y] = meshgrid(x,y);

% Defining velocity
global speed discomfort cost x_velocity y_velocity;

% Space Interval
global h;
h = Lx/(Nx-6);

% Total Time
t_total = 20;

% Flux in X and Y direction
global fx_negative fx_positive fy_negative fy_positive;
fx_positive = zeros(Ny,Nx);
fy_positive = zeros(Ny,Nx);
fx_negative = zeros(Ny,Nx);
fy_negative = zeros(Ny,Nx);

% --- Obstacle modeling ----%
global obstacle;
obstacle = false(Ny,Nx);
obstacle(101:107,31:37) = true; % Pillar
```

```

%-----Initial Condition-----%
% Zero Density initial condition
rho_zeros = zeros(Ny,Nx);

rho = rho_zeros;

%--Function to compute Velocity using Density
function speed = makeSpeed(rho)
    global Nx Ny;
    speed = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            speed(j,i) = 2*(1 - rho(j,i)/10);
        end
    end
end

%--Function to compute Discomfort using Density
function discomfort = makeDiscomfort(rho)
    global Nx Ny;
    discomfort = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            discomfort(j,i) = 0.002*rho(j,i)*rho(j,i);
        end
    end
end

%--Function to compute Cost function using Speed and Discomfort
function cost = makeCost(speed,discomfort)
    global Nx Ny;
    cost = zeros(Ny,Nx);
    for i = 1:1:Nx
        for j = 1:1:Ny
            cost(j,i) = 1/speed(j,i) + discomfort(j,i);
        end
    end
end

speed = makeSpeed(rho);
discomfort = makeDiscomfort(rho);
cost = makeCost(speed,discomfort);

% Plotting Speed
figure(1)
contourf(X,Y,speed,'LineStyle', 'none');

```

Warning: Contour not rendered for constant ZData

```

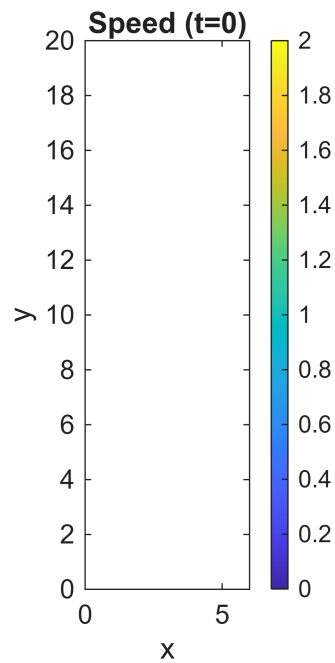
title('Speed (t=0)');

```

```

clim([0,2]);
axis equal tight;
colorbar;
xlabel('x'); ylabel('y');

```



```

% Plotting Discomfort
figure(2)
contourf(X,Y,discomfort,'LineStyle', 'none');

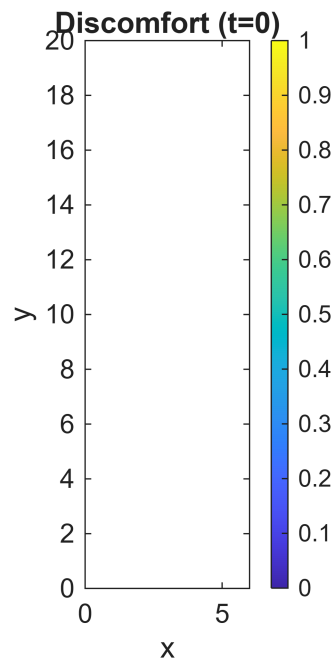
```

Warning: Contour not rendered for constant ZData

```

title('Discomfort (t=0)');
axis equal tight;
colorbar;
xlabel('x'); ylabel('y');

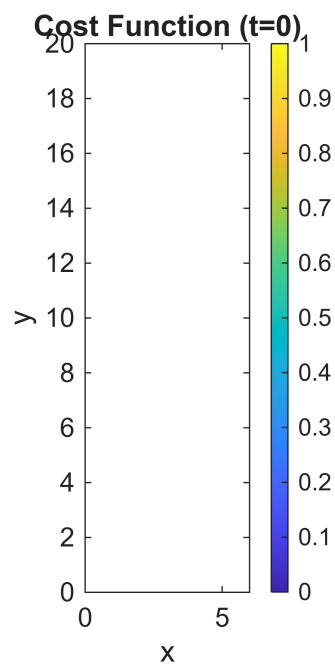
```



```
% Plotting Cost Function
figure(3);
contourf(X,Y,cost,'LineStyle', 'none');
```

Warning: Contour not rendered for constant ZData

```
title('Cost Function (t=0)');
axis equal tight;
colorbar;
xlabel('x'); ylabel('y');
```

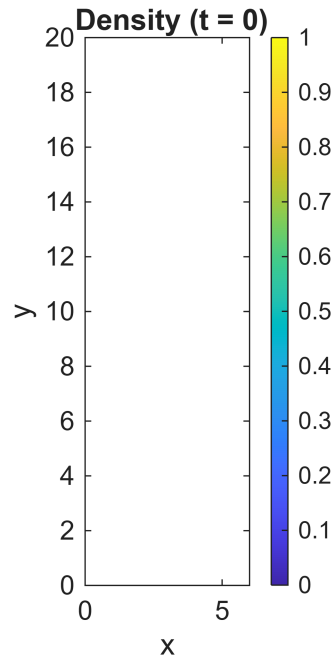


```
% Plotting the Initial Condition
```

```
figure(4);
contourf(X,Y,rho,'LineStyle','none');
```

Warning: Contour not rendered for constant ZData

```
title(' Density (t = 0)');
clim([0,1]);
axis equal tight;
colorbar;
xlabel('x'); ylabel('y');
```



```
% Function that will calculate the flux depending on the u,v and Density.
% X_velocity and y_velocity will be found using the potential.
function flux = makeFlux(rho,x_velocity,y_velocity)
    global Nx Ny;

    % Initializing Flux
    flux_x = zeros(Ny,Nx);
    flux_y = zeros(Ny,Nx);
    flux = zeros(2, Ny, Nx);

    for i = 1:1:Nx
        for j = 1:1:Ny

            flux_x(j,i) = rho(j,i) * x_velocity(j,i);
            flux_y(j,i) = rho(j,i) * y_velocity(j,i);

            flux(1,j,i) = flux_x(j,i);
            flux(2,j,i) = flux_y(j,i);

        end
    end
end
```

```

% This function will take the range of the i index and the j value where
% the flux has to be integrated and flow rate has to be computed
function flow_rate = makeFlowRate(y_velocity,rho,i_max,i_min,j)
    global h;
    flow_rate = 0;
    for i = i_min:1:i_max
        flow_rate = flow_rate + y_velocity(j,i) * rho(j,i) * h;
    end
end

% This is the variable that will store the exit flux
exit_flux = [];
% This will store the flux through the region A (Left side of the Pillar)
region_A_flux = [];
% This is the flux through the left side of the pillar
region_B_flux = [];
% This is the array which stores the time
time = [];

%--Pre allocating the memory
rho_n = zeros(Ny,Nx);
rho_1 = zeros(Ny,Nx);
rho_2 = zeros(Ny,Nx);
rho_next = zeros(Ny,Nx);

% Stopping condition
stop_low = -1;
stop_high = 11;

% This will be used for plotting
x_plot = linspace(0, Lx, Nx-6);
y_plot = linspace(0, Ly, Ny-6);

%-----MAIN SOLVER-----%
t = 0;
step = 1; % For plotting at regular interval

% Video Capturing
v = VideoWriter('New_Problem.avi'); % AVI file

```

Warning: No video frames were written to this file. The file may be invalid.

```

v.FrameRate = 30; % optional
open(v);

while t < t_total

    % Computing Cost For the density rho
    speed = makeSpeed(rho);

```

```

discomfort = makeDiscomfort(rho);
cost = makeCost(speed,discomfort);

%-- Calculate Initial Phi guess using Lower order Fast Sweep
phi_first_order = First_order_Godunov_Fast_Sweep_Algorithm(cost,h,INFINITE);
%-- Calculate The final phi using higher order scheme and above initial guess
phi =
weno_fast sweep(cost,h,eta,error_eta,INFINITE,phi_first_order,iteration_error);

% Extrapolation of the phi values
% Bottom Edge (3 Ghost Points)
phi(3,:) = phi(4,:);
phi(2,:) = phi(3,:);
phi(1,:) = phi(2,:);
% Top Edge (3 Ghost Points)
phi(Ny-2,:) = phi(Ny-3,:);
phi(Ny-1,:) = phi(Ny-2,:);
phi(Ny,:) = phi(Ny-1,:);
% Right Edge (3 Ghost Points)
phi(:,Nx-2) = phi(:,Nx-3);
phi(:,Nx-1) = phi(:,Nx-2);
phi(:,Nx) = phi(:,Nx-1);
% Left Edge (3 Ghost Points)
phi(:,3) = phi(:,4);
phi(:,2) = phi(:,3);
phi(:,1) = phi(:,2);

%--Calculate the Components of Velocity
[phi_x, phi_y] = gradient(phi, h, h); % partial derivatives of phi in x and y
mag_phi = sqrt(phi_x.^2 + phi_y.^2);
mag_phi = max(mag_phi, eta); % To be safe with denominator

for i = 1:1:Nx
    for j = 1:1:Ny
        x_velocity(j,i) = speed(j,i) * (- phi_x(j,i))/mag_phi(j,i);
        y_velocity(j,i) = speed(j,i) * (- phi_y(j,i))/mag_phi(j,i);
    end
end

% Find the Flux at region A and B and exit and time and store it in the array
region_B_flux(step) = makeFlowRate(y_velocity,rho,64,37,104);
region_A_flux(step) = makeFlowRate(y_velocity,rho,31,4,104);
exit_flux(step) = makeFlowRate(y_velocity,rho,54,14,Ny-3);
time(step) = t;

% Updating the delta_t with proper CFL condition (Stochastic paper)
u_max = max(abs(x_velocity(:)));
v_max = max(abs(y_velocity(:)));
%dt = CFL / ( ( u_max/h ).^(5/3) + ( v_max/h ).^(5/3) + 1e-12 );

```

```

dt = CFL * h / ( ( u_max ) + ( v_max ) + 1e-12 );

% -- Making rho_n and RHS of rho_n
rho_n = rho;

semi_discritze_form_rho_n = makeRHS(rho_n,t);

% Check for the negative rho_n values
if any(rho_n(:) < stop_low)
    fprintf('step = %d\n',step);
    error('Negative density detected at rho_n!');
elseif any(rho_n(:) > stop_high)
    fprintf('step = %d\n',step);
    error('Density more than 10 detected at rho_n!');
end

% -- Extrapolation (My Intuition)
% Left Edge      (3 Ghost Points)
rho_n(:,3) = rho_n(:,4);
rho_n(:,2) = rho_n(:,3);
rho_n(:,1) = rho_n(:,2);
% Bottom Edge    (3 Ghost Points)
rho_n(3,:) = rho_n(4,:);
rho_n(2,:) = rho_n(3,:);
rho_n(1,:) = rho_n(2,:);
% Top Edge       (3 Ghost Points)
rho_n(Ny-2,14:54) = 0; % This is the staircase and the outflow of the platform
rho_n(Ny-1,:) = rho_n(Ny-2,:);
rho_n(Ny,:) = rho_n(Ny-1,:);
% Right Edge     (3 Ghost Points)
rho_n(:,Nx-2) = rho_n(:,Nx-3);
rho_n(:,Nx-1) = rho_n(:,Nx-2);
rho_n(:,Nx) = rho_n(:,Nx-1);

% ----- Making the rho_1 and then RHS of rho_1 ----- %
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_1(j,i) = rho_n(j,i) + dt * semi_discritze_form_rho_n(j-3,i-3);
        else
            rho_1(j,i) = 0;
        end
    end
end

% -- Extrapolation (My Intuition)
% Left Edge      (3 Ghost Points)

```



```

rho_1(:,3) = rho_1(:,4);
rho_1(:,2) = rho_1(:,3);
rho_1(:,1) = rho_1(:,2);
% Bottom Edge    (3 Ghost Points)
rho_1(3,:) = rho_1(4,:);
rho_1(2,:) = rho_1(3,:);
rho_1(1,:) = rho_1(2,:);
% Top Edge       (3 Ghost Points)
rho_1(Ny-2,14:54) = 0; % This is the staircase and the outflow of the platform
rho_1(Ny-1,:) = rho_1(Ny-2,:);
rho_1(Ny,:) = rho_1(Ny-1,:);
% Right Edge     (3 Ghost Points)
rho_1(:,Nx-2) = rho_1(:,Nx-3);
rho_1(:,Nx-1) = rho_1(:,Nx-2);
rho_1(:,Nx) = rho_1(:,Nx-1);

% -- RHS of discretized form using rho_1
semi_discretize_form_rho_1 = makeRHS(rho_1,t);

% Check for the negative rho_1 values
if any(rho_n(:) < stop_low)
    fprintf('step = %d\n',step);
    error('Negative density detected at rho_1!');
elseif any(rho_n(:) > stop_high)
    fprintf('step = %d\n',step);
    error('Density more than 10 detected at rho_1!');
end

%----- Making the RHO_2 and RHS of RHO_2 -----%
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_2(j,i) = 3/4 * rho_n(j,i) + 1/4 * (rho_1(j,i) + dt *
semi_discretize_form_rho_1(j-3,i-3));
        else
            rho_2(j,i) = 0;
        end
    end
end

% -- Extrapolation (My Intuition)
% Left Edge     (3 Ghost Points)
rho_2(:,3) = rho_2(:,4);
rho_2(:,2) = rho_2(:,3);
rho_2(:,1) = rho_2(:,2);
% Bottom Edge   (3 Ghost Points)

```

```

rho_2(3,:) = rho_2(4,:);
rho_2(2,:) = rho_2(3,:);
rho_2(1,:) = rho_2(2,:);
% Top Edge      (3 Ghost Points)
rho_2(Ny-2,14:54) = 0; % This is the staircase and the outflow of the platform
rho_2(Ny-1,:) = rho_2(Ny-2,:);
rho_2(Ny,:) = rho_2(Ny-1,:);
% Right Edge    (3 Ghost Points)
rho_2(:,Nx-2) = rho_2(:,Nx-3);
rho_2(:,Nx-1) = rho_2(:,Nx-2);
rho_2(:,Nx) = rho_2(:,Nx-1);

% -- RHS of discretized form using rho_2
semi_discritze_form_rho_2 = makeRHS(rho_2,t);

% Check for the negative rho_2 values
if any(rho_n(:) < stop_low)
    fprintf('step = %d\n',step);
    error('Negative density detected at rho_2!');
elseif any(rho_n(:) > stop_high)
    fprintf('step = %d\n',step);
    error('Density more than 10 detected at rho_2!');
end

%-----Making the rho_next-----%

for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        if(~obstacle(j,i))
            rho_next(j,i) = 1/3 * rho_n(j,i) + 2/3 * (rho_2(j,i) + dt *
semi_discritze_form_rho_2(j-3,i-3));
        else
            rho_next(j,i) = 0;
        end
    end
end

% -- Extrapolation (My Intuition)
% Left Edge      (3 Ghost Points)
rho_next(:,3) = rho_next(:,4);
rho_next(:,2) = rho_next(:,3);
rho_next(:,1) = rho_next(:,2);
% Bottom Edge    (3 Ghost Points)
rho_next(3,:) = rho_next(4,:);
rho_next(2,:) = rho_next(3,:);
rho_next(1,:) = rho_next(2,:);
% Top Edge      (3 Ghost Points)
rho_next(Ny-2,14:54) = 0; % This is the staircase and the outflow of the
platform
rho_next(Ny-1,:) = rho_next(Ny-2,:);

```

```

rho_next(Ny,:) = rho_next(Ny-1,:);
% Right Edge    (3 Ghost Points)
rho_next(:,Nx-2) = rho_next(:,Nx-3);
rho_next(:,Nx-1) = rho_next(:,Nx-2);
rho_next(:,Nx) = rho_next(:,Nx-1);

% Substituting back the rho value
rho = rho_next;

%--Check if any of the density values is equal to zero
if any(rho_n(:) < stop_low)
    fprintf('step = %d\n',step);
    error('Negative density detected at rho_next!');
elseif any(rho_n(:) > stop_high)
    fprintf('step = %d\n',step);
    error('Density more than 10 detected at rho_next!');
end

% Here I could plot the flux at each time step.

%--Plotting the rho_next;
if mod(step, 2) == 0

    figure(6);

    contourf(x_plot, y_plot, rho(4:Ny-3,4:Nx-3), 150, 'LineStyle', 'none');

    hold on;

    contour(x_plot, y_plot, obstacle(4:Ny-3,4:Nx-3), [0.5 0.5], 'k',
'LineWidth', 2);

    hold off;

    xlim([0 Lx]);
    ylim([0 Ly]);

    colorbar;
    clim([0 10]);

    title(sprintf('Density (rho) at t = %.2f', t));
    xlabel('x');
    ylabel('y');

    axis equal tight;
    drawnow;

end

```

```

    % Updating the time
    t = t + dt;

    step = step + 1;
end
% Close the Video
close(v);

% Now plot the time vs the Exit flux and same for the region A and region B
figure;

plot(time, region_A_flux, 'LineWidth', 1);
hold on;

plot(time, region_B_flux, 'LineWidth', 1);

plot(time, exit_flux, 'LineWidth', 1);

xlabel('Time');
ylabel('Flow Rate');
title('Flow Rate vs Time');

legend('Region A', 'Region B', 'Exit');
grid on;

```

First order Godunov Fast Sweep Algorithm (Older Version)

```

function phi = First_order_Godunov_Fast_Sweep_Algorithm(c,h,INFINITE)
    global Nx Ny obstacle;
    phi = INFINITE*ones(Ny, Nx);    % Initial potential field

    % Exit Boundary (Top Boundary of the staircase)
    phi(Ny-3,14:54) = 0;

    %-----
    % Fast Sweeping Iterations
    %-----
    for sweep = 1:20

        % Sweep 1:
        for i = 4:Nx-2
            for j = 4:Ny-2
                if ~obstacle(j, i)
                    phi = update_phi(phi, c, i, j, h);
                else
                    phi(j,i) = INFINITE;
                end
            end
        end
    end
end

```

```

% Sweep 2:
for i = Nx-1:-1:2
    for j = 2:Ny-1
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Sweep 3:
for i = Nx-1:-1:2
    for j = Ny-1:-1:2
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Sweep 4:
for i = 2:Nx-1
    for j = Ny-1:-1:2
        if ~obstacle(j, i)
            phi = update_phi(phi, c, i, j, h);
        else
            phi(j,i) = INFINITE;
        end
    end
end

% Extrapolation of the phi values

% Bottom Edge    (3 Ghost Points)
phi(3,:) = phi(4,:);
phi(2,:) = phi(3,:);
phi(1,:) = phi(2,:);

% Top Edge       (3 Ghost Points)
phi(Ny-2,:) = phi(Ny-3,:);
phi(Ny-1,:) = phi(Ny-2,:);
phi(Ny,:) = phi(Ny-1,:);

% Right Edge     (3 Ghost Points)
phi(:,Nx-2) = phi(:,Nx-3);
phi(:,Nx-1) = phi(:,Nx-2);
phi(:,Nx) = phi(:,Nx-1);

```

```

    % Left Edge      (3 Ghost Points)
    phi(:,3) = phi(:,4);
    phi(:,2) = phi(:,3);
    phi(:,1) = phi(:,2);
end
end

% Subfunction: Update one grid point using Godunov scheme

function phi = update_phi(phi, c, i, j, dx)
    a = min(phi(j, i-1), phi(j, i+1)); % x-direction neighbors
    b = min(phi(j-1, i), phi(j+1, i)); % y-direction neighbors
    c_i = c(j, i); % Local inverse speed
    if abs(a - b) >= c_i * dx
        phi(j, i) = min(a, b) + c_i * dx;
    else
        inside = 2 * (c_i * dx)^2 - (a - b)^2;
        if inside >= 0
            phi(j, i) = (a + b + sqrt(inside)) / 2;
        end
    end
end
end
end

```

WENO FAST SWEEP

```

function phi =
weno_fast sweep(c,h,eta,error_eta,INFINITE,initial_guess,iteration_error)

    global Nx Ny obstacle;

    % Phi is initialized with infinity
    phi = initial_guess;
    phi_old = INFINITE * ones(size(phi));

    % Iterations
    iterations = 0;
    loop_safety = 0;

    % Iterations
    while (sum(abs(phi(:)-phi_old(:))) > error_eta && loop_safety<1000)

        %fprintf("Iteration = %d ",iterations);
        err = sum(abs(phi(:)-phi_old(:)));
        %fprintf("Error = %0.10f\n",err);

        phi_old = phi;
        iterations = iterations + 1;
        iteration_error(iterations) = err;
        loop_safety = loop_safety + 1;
    end
end

```

```

for sweep = 1:4
    switch sweep
        case 1
            ix = 4:1:Nx-3; jy = 4:1:Ny-3;
        case 2
            ix = 4:1:Nx-3; jy = Ny-3:-1:4;
        case 3
            ix = Nx-3:-1:4; jy = 4:1:Ny-3;
        case 4
            ix = Nx-3:-1:4; jy = Ny-3:-1:4;
    end

    for i = ix
        for j = jy
            %if the point is outside the obstacle
            if(~obstacle(j,i))
                %-----Calculating the phix_min-----_%

                r_back = (eta + (phi(j,i) - 2*phi(j,i-1) + phi(j,i-2))^2)/
(eta + (phi(j,i+1) - 2*phi(j,i) + phi(j,i-1))^2);
                r_front = (eta + (phi(j,i) - 2*phi(j,i+1) + phi(j,i+2))^2)/
(eta + (phi(j,i+1) - 2*phi(j,i) + phi(j,i-1))^2);

                % Calculate the w
                w_front = 1/(1+2*(r_front^2));
                w_back = 1/(1+2*(r_back^2));

                % Dell phi by Dell x plus and minus
                phix_minus = (1-w_back)*((phi(j,i+1)-phi(j,i-1))/(2*h)) +
w_back*((3*phi(j,i) - 4*phi(j,i-1) + phi(j,i-2))/(2*h));
                phix_plus = (1-w_front)*((phi(j,i+1)-phi(j,i-1))/(2*h)) +
w_front*((-3*phi(j,i)+4*phi(j,i+1)-phi(j,i+2))/(2*h));

                phix_min = min((phi(j,i) - h*phix_minus),(phi(j,i) +
h*phix_plus));

                %-----calculating the phiy_min-----_%

                r_back = (eta + (phi(j,i) - 2*phi(j-1,i) + phi(j-2,i))^2) /
(eta + (phi(j+1,i) - 2*phi(j,i) + phi(j-1,i))^2);
                r_front = (eta + (phi(j,i) - 2*phi(j+1,i) +
phi(j+2,i))^2) / (eta + (phi(j+1,i) - 2*phi(j,i) + phi(j-1,i))^2);

                % Calculate the w
                w_front = 1 / (1 + 2*(r_front^2));
                w_back = 1 / (1 + 2*(r_back^2));

                % Dell phi by Dell x plus and minus

```

```

        phiy_minus = (1-w_back) * ((phi(j+1,i)-phi(j-1,i))/(2*h)) +
w_back * ((3*phi(j,i) - 4*phi(j-1,i) + phi(j-2,i))/(2*h));
        phiy_plus = (1-w_front) * ((phi(j+1,i) - phi(j-1,i))/(2*h))
+ w_front * ((-3*phi(j,i) + 4*phi(j+1,i) - phi(j+2,i))/(2*h));

        phiy_min = min((phi(j,i) - h*phiy_minus), (phi(j,i) +
h*phiy_plus));

        if abs(phix_min - phiy_min) >= c(j,i)*h
            phi(j,i) = min(phiy_min,phix_min) + c(j,i)*h;
        else
            phi(j,i) = ((phix_min + phiy_min) + (2*(c(j,i)*h)^2 -
(phix_min - phiy_min)^2)^0.5)/2;
        end

        % Zero potential at region of the top boundary with
        % the top boundary as staircase outflow.
        if(j == Ny-3)
            if(i >= 14 && i <= 54)
                phi(j,i) = 0;
            end
        end

        % If the point is inside the obstacle
        else
            phi(j,i) = INFINITE;
        end
    end
end
end
end
end
end
end

```

RHS of Semi Discretize Form

```

function semi_discrete_form = makeRHS(rho,t)
    global Nx Ny x_velocity y_velocity h obstacle fx_positive fx_negative
    fy_positive fy_negative;
    semi_discrete_form = zeros(Ny-6,Nx-6);

    %--Flux for the given rho distribution
    flux = makeFlux(rho,x_velocity,y_velocity);
    flux_x = squeeze(flux(1,:,:));
    flux_y = squeeze(flux(2,:,:));

    % Calculating the maximum speed (multiplying by 1.5 as suggested)
    alpha_x = max(abs(x_velocity(:))) * 1.5;
    alpha_y = max(abs(y_velocity(:))) * 1.5;

```



```

% -- Calculating the value of f+ and f- at {i+2,i+1,i,i-1,i-2,i-3} -- %
fx_negative_whole = zeros(Ny,Nx);
fx_positive_whole = zeros(Ny,Nx);
fy_negative_whole = zeros(Ny,Nx);
fy_positive_whole = zeros(Ny,Nx);

for j = 1:1:Ny
    for i=1:1:Nx
        fx_positive_whole(j,i) = 1/2 * (flux_x(j,i) + alpha_x*rho(j,i));
        fx_negative_whole(j,i) = 1/2 * (flux_x(j,i) - alpha_x*rho(j,i));
        fy_positive_whole(j,i) = 1/2 * (flux_y(j,i) + alpha_y*rho(j,i));
        fy_negative_whole(j,i) = 1/2 * (flux_y(j,i) - alpha_y*rho(j,i));
    end
end

% -- Computing the RHS of Semi Discrete form.
for i = 4:1:Nx-3
    for j = 4:1:Ny-3
        % compute only at the points where obstacle doesn't exist
        if(~obstacle(j,i))
            %--X--%

            % fx{+}_{i+1/2,j} - Given in the paper
            fx_plus_x_half_forward = stencil( ...
                fx_positive_whole(j,i-2), ...
                fx_positive_whole(j,i-1), ...
                fx_positive_whole(j,i), ...
                fx_positive_whole(j,i+1), ...
                fx_positive_whole(j,i+2));

            % fx{+}_{i-1/2,j} - By substituting i -> (i-1) in fx{+}_{i+1/2,j}
            fx_plus_x_half_backward = stencil( ...
                fx_positive_whole(j,i-3), ...
                fx_positive_whole(j,i-2), ...
                fx_positive_whole(j,i-1), ...
                fx_positive_whole(j,i), ...
                fx_positive_whole(j,i+1));

            % fx{-}_{i+1/2,j} ( mirror of fx{+}_{i+1/2,j} )
            fx_minus_x_half_forward = stencil( ...
                fx_negative_whole(j,i+3), ...
                fx_negative_whole(j,i+2), ...
                fx_negative_whole(j,i+1), ...
                fx_negative_whole(j,i), ...
                fx_negative_whole(j,i-1));

            % fx{-}_{i-1/2,j} ( mirror of fx{+}_{i-1/2,j} )
            fx_minus_x_half_backward = stencil( ...

```

```

        fx_negative_whole(j,i+2), ...
        fx_negative_whole(j,i+1), ...
        fx_negative_whole(j,i), ...
        fx_negative_whole(j,i-1), ...
        fx_negative_whole(j,i-2));

%--Y--%

% fy{+}_[i,j+1/2]
fy_plus_y_half_forward = stencil( ...
    fy_positive_whole(j-2,i), ...
    fy_positive_whole(j-1,i), ...
    fy_positive_whole(j,i), ...
    fy_positive_whole(j+1,i), ...
    fy_positive_whole(j+2,i));

% fy{+}_[i,j-1/2]
fy_plus_y_half_backward = stencil( ...
    fy_positive_whole(j-3,i), ...
    fy_positive_whole(j-2,i), ...
    fy_positive_whole(j-1,i), ...
    fy_positive_whole(j,i), ...
    fy_positive_whole(j+1,i));

% fy{-}_[i,j+1/2]
fy_minus_y_half_forward = stencil( ...
    fy_negative_whole(j+3,i), ...
    fy_negative_whole(j+2,i), ...
    fy_negative_whole(j+1,i), ...
    fy_negative_whole(j,i), ...
    fy_negative_whole(j-1,i));

% fy{-}_[i,j-1/2]
fy_minus_y_half_backward = stencil( ...
    fy_negative_whole(j+2,i), ...
    fy_negative_whole(j+1,i), ...
    fy_negative_whole(j,i), ...
    fy_negative_whole(j-1,i), ...
    fy_negative_whole(j-2,i));

fx_positive_half = fx_plus_x_half_forward + fx_minus_x_half_forward;
fx_negative_half = fx_plus_x_half_backward +
fx_minus_x_half_backward;
fy_positive_half = fy_plus_y_half_forward + fy_minus_y_half_forward;
fy_negative_half = fy_plus_y_half_backward +
fy_minus_y_half_backward;

%--Boundary Condition -- %

```

```

% Left Boundary (Flux need not be computed)
if(i == 4)
    if((j >= 13 && j <= 20) || (j >= 188 && j <= 195))
        % Y flux is said to be zero at the inflow
        fy_negative_half = 0;
        % Unsteady Boundary Condition (for the x flux)
        fx_negative_half = 4;
    else
        % This is the wall of the Left boundary
        fx_negative_half = 0;
    end
end

% Bottom Wall (Normal Flux on the boundary is zero)
if(j == 4)
    fy_negative_half = 0;
end

% Top Wall (Normal Flux on the boundary is zero)
if(j == Ny-3)
    if(i <= 13 || i >= 55)
        fy_positive_half = 0;
    end
end

% Right Boundary (Normal Flux at non-exit is zero)
if(i == Nx-3)
    % This is the walls of the right boundary
    % fx_positive_half = 0;
    if((j >= 13 && j <= 20) || (j >= 188 && j <= 195))
        % Y flux is said to be zero at the inflow
        fy_negative_half = 0;
        % Unsteady Boundary Condition (for the x flux)
        fx_negative_half = 4;
    else
        % This is the wall of the Left boundary
        fx_positive_half = 0;
    end
end

% -- Obstacle Boundary --%
% Pillar Obstacle
if(j >= 101 && j <= 107)
    if(i == 30)
        fx_positive_half = 0; % Left Boundary
    elseif(i == 38)
        fx_negative_half = 0; % Right Boundary
    end
end
% Pillar Obstacle

```

```

        if(i <= 37 && i >= 31)
            if(j == 100)
                fy_positive_half = 0; % Bottom Boundary
            elseif(j == 108)
                fy_negative_half = 0; % Top Boundary
            end
        end
    end

    fx_negative(j,i) = fx_negative_half;
    fx_positive(j,i) = fx_positive_half;
    fy_negative(j,i) = fy_negative_half;
    fy_positive(j,i) = fy_positive_half;

    % Computing RHS of Semi Discretized form
    semi_discrete_form(j-3,i-3) = - 1/h * (fx_positive_half -
fx_negative_half) - 1/h * (fy_positive_half - fy_negative_half);

    else
        % Else when there is an obstacle
    end
end
end
end
end

```

WENO Advection Stencil

```

% To compute f half. Order of input matters
function f = stencil(f1,f2,f3,f4,f5)

    eta = 10^-6;

    % Compute S values
    s_1 = ((1/3) * f1) + ((-7/6) * f2) + ((11/6) * f3);
    s_2 = ((-1/6) * f2) + ((5/6) * f3) + ((1/3) * f4);
    s_3 = ((1/3) * f3) + ((5/6) * f4) + ((-1/6) * f5);

    % Compute Beta
    beta_1 = 13/12 * (f1 - 2*f2 + f3)^2 + 1/4 * (f1 - 4*f2 + 3*f3)^2;
    beta_2 = 13/12 * (f2 - 2*f3 + f4)^2 + 1/4 * (f2 - f4)^2;
    beta_3 = 13/12 * (f3 - 2*f4 + f5)^2 + 1/4 * (3*f3 - 4*f4 + f5)^2;

    % Defining Gamma
    gamma_1 = 1/10;
    gamma_2 = 3/5;
    gamma_3 = 3/10;

    % Computing weights bar
    w_1_dash = ( gamma_1 ) ./ ((eta + beta_1)^2);
    w_2_dash = ( gamma_2 ) ./ ((eta + beta_2)^2);

```

```
w_3_dash = ( gamma_3 ) ./ ((eta + beta_3)^2);

% Computing Weights
w_1 = ( w_1_dash ) / (w_1_dash + w_2_dash + w_3_dash);
w_2 = ( w_2_dash ) / (w_1_dash + w_2_dash + w_3_dash);
w_3 = ( w_3_dash ) / (w_1_dash + w_2_dash + w_3_dash);

% Computing Flux
f = w_1 * s_1 + w_2 * s_2 + w_3 * s_3;
end
```